

MeridianBank SQL Analytics Project

Project Overview

Purpose: This project uses a simulated retail banking database to answer business questions about customer relationships, account balances, cash flow, branch performance, merchant/card spending, risk signals, and data quality checks.

Database: MeridianBank, a six-table SQLite model with branches, merchants, customers, accounts, account holders, and transactions.

Analyst framing: Queries are organized by business use case, with comments added to explain what each query produces and why it matters.

Portfolio note: Because the database is simulated, the insights are illustrative examples of how the SQL outputs would be interpreted in a real analyst workflow.

Skills Demonstrated

Skill Area	Examples in Project
SQL fundamentals	SELECT, WHERE, ORDER BY, aliases, calculated fields
Relational modeling	Primary keys, foreign keys, many-to-many relationships, normalized tables
Aggregation	COUNT, SUM, AVG, GROUP BY, HAVING-style summaries
Conditional logic	CASE statements and conditional aggregation
Joins	INNER JOIN, LEFT JOIN, multi-table joins across customers/accounts/transactions
Window functions	RANK, NTILE, LAG, running/cumulative analysis
Business analysis	Branch performance, customer value, merchant spend, risk flags, data quality checks

Data Model Summary

Table	Business Meaning
branches	Bank locations where customers and accounts are assigned.
merchants	Businesses where card transactions occur.
customers	Customer profiles and home branch assignments.
accounts	Bank accounts, account types, balances, statuses, and branch assignment.
account_holders	Junction table connecting customers to accounts, enabling joint accounts.
transactions	Ledger of account activity, including salaries, deposits, withdrawals, transfers, card payments, fees, and interest.

Customer & Account Overview

Query 1 - Account Holder Directory

Business purpose: Show each account, the account type, the customer attached to it, and whether the customer is a primary or joint holder.

SQL skills demonstrated: JOINS, many-to-many relationship mapping, aliases

```
-- Shows every account holder relationship, including account type and holder role.
SELECT
    AH.account_id,
    A.account_type,
    AH.customer_id,
    C.first_name,
    C.last_name,
    AH.role
FROM account_holders AS AH
JOIN customers AS C
    ON AH.customer_id = C.customer_id
JOIN accounts AS A
    ON AH.account_id = A.account_id;
```

Query 2 - Total Balance by Branch

Business purpose: Summarize the total balance held at each branch.

SQL skills demonstrated: JOINS, SUM, GROUP BY

```
-- Shows total account balance held at each branch.
SELECT
    B.branch_id,
    SUM(A.current_balance) AS total_balance
FROM branches AS B
JOIN accounts AS A
    ON B.branch_id = A.branch_id
GROUP BY B.branch_id;
```

Query 3 - Test Branch Insert and Branch Review

Business purpose: Add a temporary test branch and review the branch table after insertion.

SQL skills demonstrated: INSERT, SELECT, DML validation

```
-- Inserts a test branch for DML practice, then reviews all branch records.
INSERT INTO branches (branch_id, branch_name, city, country, opened_date)
VALUES (4, 'Test Branch', 'Test City', 'TC', '2010-01-01');

SELECT *
FROM branches;
```

Query 4 - Account Cash Flow Summary

Business purpose: For every account, calculate total transactions, money in, money out, and net movement.

SQL skills demonstrated: Conditional aggregation, CASE, SUM, COUNT, GROUP BY

```
-- Summarizes transaction activity for every account.
-- money_in captures positive transactions; money_out captures negative transactions.
SELECT
    A.account_id,
    A.account_type,
    COUNT(T.transaction_id) AS transaction_count,
    SUM(CASE WHEN T.amount > 0 THEN T.amount ELSE 0 END) AS money_in,
    SUM(CASE WHEN T.amount < 0 THEN T.amount ELSE 0 END) AS money_out,
    SUM(T.amount) AS net_change
FROM accounts AS A
JOIN transactions AS T
    ON A.account_id = T.account_id
GROUP BY A.account_id, A.account_type
ORDER BY net_change DESC;
```

Key insights from sample data:

- Account 6 produced the highest net change in the sample data at 9150.00, driven by recurring salary inflows despite large transfer and withdrawal activity.
- The credit account shows a negative net change of -150.00, making it useful for demonstrating balance-risk monitoring.

Query 5 - Customer Account Overview

Business purpose: Show how many accounts each customer holds, their total balance, and whether they hold a joint account.

SQL skills demonstrated: JOINS, COUNT, SUM, MAX, CASE, GROUP BY

```
-- Shows a customer-level account summary, including a joint account flag.
SELECT
  C.customer_id,
  C.first_name,
  C.last_name,
  COUNT(AH.account_id) AS total_accounts,
  ROUND(SUM(A.current_balance), 2) AS total_balance,
  MAX(CASE WHEN AH.role = 'joint' THEN 1 ELSE 0 END) AS has_joint_account
FROM customers AS C
JOIN account_holders AS AH
  ON C.customer_id = AH.customer_id
JOIN accounts AS A
  ON AH.account_id = A.account_id
GROUP BY C.customer_id, C.first_name, C.last_name
ORDER BY total_balance DESC;
```

Key insights from sample data:

- Marco Rossi and Sofia Almeida show the highest total balances because they are attached to joint savings activity in addition to their other accounts.
- This view highlights why many-to-many account relationships matter: customer-level balances can change depending on how joint accounts are treated.

Query 6 - Customer Account Detail Rows

Business purpose: Display each customer-account row without aggregating, useful for validating the customer overview query.

SQL skills demonstrated: JOINS, row-level validation, ORDER BY

```
-- Shows the row-level detail behind each customer's account holdings.
SELECT
  C.customer_id,
  C.first_name,
  C.last_name,
  AH.account_id,
  A.current_balance,
  CASE WHEN AH.role = 'joint' THEN 1 ELSE 0 END AS is_joint_account
FROM customers AS C
JOIN account_holders AS AH
  ON C.customer_id = AH.customer_id
JOIN accounts AS A
  ON AH.account_id = A.account_id
ORDER BY C.customer_id, AH.account_id;
```

Branch Performance & Cash Flow

Query 7 - Branch Cash Flow Overview

Business purpose: Show cash flow by branch, including card payment outflows, money in, money out, and net cash flow.

SQL skills demonstrated: Conditional aggregation, joins, SUM, GROUP BY

```
-- Shows branch-level cash flow and separates card spend from overall inflows/outflows.
SELECT
    B.branch_id,
    B.branch_name,
    SUM(CASE WHEN T.txn_type = 'card_payment' AND T.amount < 0 THEN T.amount ELSE 0 END) AS card_payment_out,
    SUM(CASE WHEN T.amount > 0 THEN T.amount ELSE 0 END) AS money_in,
    SUM(CASE WHEN T.amount < 0 THEN T.amount ELSE 0 END) AS money_out,
    SUM(T.amount) AS cash_flow
FROM branches AS B
JOIN accounts AS A
    ON B.branch_id = A.branch_id
JOIN transactions AS T
    ON A.account_id = T.account_id
GROUP BY B.branch_id, B.branch_name
ORDER BY cash_flow DESC;
```

Key insights from sample data:

- Kirchberg Branch has the highest overall cash flow in the sample data, with total account balances/net flow of 28198.85.
- Separating card payment outflows from total inflows/outflows makes the branch summary more useful than a single net number.

Query 8 - Month-over-Month Branch Credits

Business purpose: Analyze whether branch inflows are increasing or decreasing month over month.

SQL skills demonstrated: Window functions, LAG, date extraction, conditional aggregation

```
-- Calculates monthly money in by branch and compares it to the previous month.
SELECT
    B.branch_id,
    B.branch_name,
    STRFTIME('%m', T.txn_date) AS month,
    SUM(CASE WHEN T.amount > 0 THEN T.amount ELSE 0 END) AS money_in,
    LAG(SUM(CASE WHEN T.amount > 0 THEN T.amount ELSE 0 END), 1)
        OVER (PARTITION BY B.branch_id ORDER BY STRFTIME('%m', T.txn_date)) AS prior_month_money_in,
    ROUND(
        SUM(CASE WHEN T.amount > 0 THEN T.amount ELSE 0 END)
        - LAG(SUM(CASE WHEN T.amount > 0 THEN T.amount ELSE 0 END), 1)
            OVER (PARTITION BY B.branch_id ORDER BY STRFTIME('%m', T.txn_date)),
        2
    ) AS month_over_month_change
FROM branches AS B
JOIN accounts AS A
    ON B.branch_id = A.branch_id
JOIN transactions AS T
    ON A.account_id = T.account_id
GROUP BY B.branch_id, B.branch_name, STRFTIME('%m', T.txn_date)
ORDER BY B.branch_id, month;
```

Key insights from sample data:

- Kirchberg increased from January to February by 1005.00, then declined by 493.70 in March.
- Esch Branch shows the sharpest slowdown, falling from 5000.00 in January inflows to 408.50 by March.

Ranking, Segmentation & Window Functions

Query 9 - Customers Ranked Within Each Branch

Business purpose: Rank customers within each branch based on account balance.

SQL skills demonstrated: RANK, PARTITION BY, joins

```
-- Ranks customers within each branch by account balance.
SELECT
    B.branch_id,
    B.branch_name,
    C.customer_id,
    C.first_name,
    C.last_name,
    A.current_balance,
    RANK() OVER (PARTITION BY B.branch_id ORDER BY A.current_balance DESC) AS branch_balance_rank
FROM customers AS C
JOIN account_holders AS AH
    ON C.customer_id = AH.customer_id
JOIN accounts AS A
    ON AH.account_id = A.account_id
JOIN branches AS B
    ON A.branch_id = B.branch_id;
```

Query 10 - Customer Activity Quartiles

Business purpose: Split customers into four groups based on total transaction volume.

SQL skills demonstrated: CTEs, ABS, SUM, NTILE, window functions

```
-- Calculates total transaction volume per customer and divides customers into four activity quartiles.
WITH TransactionVolumes AS (
    SELECT
        C.customer_id,
        C.first_name,
        C.last_name,
        SUM(ABS(T.amount)) AS total_transaction_volume
    FROM customers AS C
    JOIN account_holders AS AH
        ON C.customer_id = AH.customer_id
    JOIN accounts AS A
        ON AH.account_id = A.account_id
    JOIN transactions AS T
        ON A.account_id = T.account_id
    GROUP BY C.customer_id, C.first_name, C.last_name
)
SELECT
    customer_id,
    first_name,
    last_name,
    total_transaction_volume,
    NTILE(4) OVER (ORDER BY total_transaction_volume DESC) AS activity_quartile
FROM TransactionVolumes;
```

Key insights from sample data:

- Marco Rossi and Sofia Almeida fall into the highest activity quartile based on total absolute transaction volume.
- Using absolute transaction amounts captures overall activity better than net flow because both deposits and withdrawals count toward customer engagement.

Merchant & Card Spending Analysis

Query 11 - Each Customer's Top Spending Category

Business purpose: Identify each customer's highest card-spending merchant category.

SQL skills demonstrated: CTEs, joins, RANK, PARTITION BY, card spend analysis

```
-- Finds each customer's top card-spending category and amount.
WITH Category_Spending AS (
    SELECT
        C.customer_id,
        C.first_name,
        C.last_name,
        M.category,
        SUM(T.amount) AS amount
    FROM customers AS C
    JOIN account_holders AS AH
        ON C.customer_id = AH.customer_id
    JOIN transactions AS T
        ON AH.account_id = T.account_id
    JOIN merchants AS M
        ON T.merchant_id = M.merchant_id
    WHERE T.txn_type = 'card_payment'
    GROUP BY C.customer_id, C.first_name, C.last_name, M.category
),
RankedCategorySpend AS (
    SELECT
        customer_id,
        first_name,
        last_name,
        category,
        amount,
        RANK() OVER (PARTITION BY customer_id ORDER BY amount ASC) AS category_rank
    FROM Category_Spending
)
SELECT
    customer_id,
    first_name,
    last_name,
    category,
    amount
FROM RankedCategorySpend
WHERE category_rank = 1;
```

Query 12 - Top Category as Share of Customer Card Spend

Business purpose: Show each customer's top spending category and what percentage of their total card spend it represents.

SQL skills demonstrated: Multiple CTEs, percentage calculation, joins, RANK

```
-- Calculates each customer's top card-spending category as a share of total card spend.
WITH Total_Spending AS (
    SELECT
        C.customer_id,
        SUM(T.amount) AS total_amount
    FROM customers AS C
    JOIN account_holders AS AH
        ON C.customer_id = AH.customer_id
    JOIN transactions AS T
        ON AH.account_id = T.account_id
    JOIN merchants AS M
        ON T.merchant_id = M.merchant_id
    WHERE T.txn_type = 'card_payment'
    GROUP BY C.customer_id
),
Category_Spending AS (
    SELECT
        C.customer_id,
        M.category,
```

```

        SUM(T.amount) AS amount
    FROM customers AS C
    JOIN account_holders AS AH
        ON C.customer_id = AH.customer_id
    JOIN transactions AS T
        ON AH.account_id = T.account_id
    JOIN merchants AS M
        ON T.merchant_id = M.merchant_id
    WHERE T.txn_type = 'card_payment'
    GROUP BY C.customer_id, M.category
),
Top_Category AS (
    SELECT *
    FROM (
        SELECT
            customer_id,
            category,
            amount,
            RANK() OVER (PARTITION BY customer_id ORDER BY amount ASC) AS rank
        FROM Category_Spending
    )
    WHERE rank = 1
)
SELECT
    TC.customer_id,
    TC.category,
    TC.amount,
    TS.total_amount,
    (ROUND((CAST(TC.amount AS FLOAT) / CAST(TS.total_amount AS FLOAT) * 100), 2) || '%') AS percentage_of_total
FROM Top_Category AS TC
JOIN Total_Spending AS TS
    ON TC.customer_id = TS.customer_id;

```

Key insights from sample data:

- Retail dominates the top category for most card-active customers in the sample dataset.
- For Clara Muller and Liam OBrien, the top category represents more than three-quarters of total card spend, showing highly concentrated spending behavior.

Risk, Anomalies & Data Quality

Query 13 - Large Outflow Anomaly Flag

Business purpose: Flag outflows that are at least twice the account's average outflow.

SQL skills demonstrated: CTEs, AVG, CASE, anomaly detection

```

-- Flags transactions where an account outflow is at least 2x that account's average outflow.
WITH Outflow AS (
    SELECT
        account_id,
        txn_date,
        amount
    FROM transactions
    WHERE amount < 0
),
AverageOutflow AS (
    SELECT
        account_id,
        ROUND(AVG(-amount), 2) AS average_outflow
    FROM Outflow
    GROUP BY account_id
)
SELECT
    AO.account_id,
    O.txn_date,
    O.amount,

```

```

    AO.average_outflow,
    ROUND((-O.amount) / AO.average_outflow, 2) AS times_average,
    CASE WHEN -O.amount >= AO.average_outflow * 2 THEN 'High' ELSE '' END AS flag
FROM AverageOutflow AS AO
JOIN Outflow AS O
    ON AO.account_id = O.account_id
WHERE -O.amount >= AO.average_outflow * 2;

```

Key insights from sample data:

- The query flags transactions that are unusual relative to each account, not just large in absolute terms.
- Account 1 has the highest relative anomaly: a 500.00 transfer equal to 3.28x its average outflow.

Query 14 - One-Day High Net Inflow Count

Business purpose: Count account-day combinations where the net movement exceeded 1000.

SQL skills demonstrated: CTE, GROUP BY, HAVING, COUNT

```

-- Counts account/date combinations where daily net movement was greater than 1000.
WITH HighNetflow AS (
    SELECT
        account_id,
        txn_date,
        SUM(amount) AS net_flow
    FROM transactions
    GROUP BY account_id, txn_date
    HAVING SUM(amount) > 1000
)
SELECT COUNT(*) AS high_netflow_days
FROM HighNetflow;

```

Query 15 - Transaction Count by Account

Business purpose: Count how many transactions each account has.

SQL skills demonstrated: COUNT, GROUP BY

```

-- Shows total number of transactions for each account.
SELECT
    account_id,
    COUNT(*) AS transaction_count
FROM transactions
GROUP BY account_id
ORDER BY transaction_count DESC;

```

Query 16 - Longest Gaps Between Transactions

Business purpose: Calculate the number of days since each account's previous transaction and rank the longest gaps.

SQL skills demonstrated: LAG, JULIANDAY, window functions, ranking

```

-- Calculates days since the previous transaction for each account and ranks the longest gaps.
WITH TransactionList AS (
    SELECT
        account_id,
        transaction_id,
        txn_date,
        LAG(txn_date, 1) OVER (PARTITION BY account_id ORDER BY transaction_id) AS prev_txn
    FROM transactions
)
SELECT
    account_id,
    transaction_id,
    txn_date,
    prev_txn,
    JULIANDAY(txn_date) - JULIANDAY(prev_txn) AS days_since_previous_txn,

```

```

RANK() OVER (
    PARTITION BY account_id
    ORDER BY JULIANDAY(txn_date) - JULIANDAY(prev_txn) DESC
) AS gap_rank
FROM TransactionList
WHERE prev_txn IS NOT NULL;

```

Query 17 - Accounts Without Holders

Business purpose: Identify accounts that do not have any customer assigned to them.

SQL skills demonstrated: LEFT JOIN, NULL filtering, data quality check

```

-- Data quality check: returns accounts with no matching holder record.
SELECT
    A.*
FROM accounts AS A
LEFT JOIN account_holders AS AH
    ON A.account_id = AH.account_id
WHERE AH.account_id IS NULL;

```

Query 18 - Monthly Net Flow by Transaction Type

Business purpose: Create a monthly overview of transaction counts and net flow by transaction type, ranked within each month.

SQL skills demonstrated: CTE, date extraction, GROUP BY, RANK

```

-- Summarizes monthly net flow by transaction type and ranks transaction types within each month.
WITH NetFlow AS (
    SELECT
        STRFTIME('%Y', txn_date) AS year,
        STRFTIME('%m', txn_date) AS month,
        COUNT(transaction_id) AS total_transactions,
        txn_type,
        SUM(amount) AS net_flow
    FROM transactions
    GROUP BY STRFTIME('%Y', txn_date), STRFTIME('%m', txn_date), txn_type
)
SELECT
    *,
    RANK() OVER (PARTITION BY month ORDER BY net_flow DESC) AS rank
FROM NetFlow;

```

Growth, Value & Customer Health Analysis

Query 19 - Cumulative Account Growth

Business purpose: Show how many accounts were opened each year and calculate the running cumulative total.

SQL skills demonstrated: Date extraction, COUNT, SUM OVER, running totals

```

-- Counts accounts opened per year and calculates cumulative account growth over time.
WITH AddedAcc AS (
    SELECT
        STRFTIME('%Y', opened_date) AS year,
        COUNT(account_id) AS added_accounts
    FROM accounts
    GROUP BY STRFTIME('%Y', opened_date)
)
SELECT
    year,
    added_accounts,
    SUM(added_accounts) OVER (ORDER BY year) AS cumulative_accounts
FROM AddedAcc;

```

Query 20 - Most Valuable Customers with Joint Account Split

Business purpose: Rank customers by estimated total balance while splitting joint account balances between holders.

SQL skills demonstrated: CASE, joins, SUM, joint account logic, ORDER BY

```
-- Estimates customer value by splitting joint account balances between account holders.
WITH RankedBalances AS (
    SELECT
        A.account_id,
        AH.customer_id,
        AH.role,
        CASE
            WHEN A.account_id IN (1, 9) THEN A.current_balance / 2
            ELSE A.current_balance
        END AS acc_balance
    FROM accounts AS A
    JOIN account_holders AS AH
        ON A.account_id = AH.account_id
)
SELECT
    RB.customer_id,
    C.first_name,
    C.last_name,
    SUM(acc_balance) AS total_balance
FROM RankedBalances AS RB
JOIN customers AS C
    ON RB.customer_id = C.customer_id
GROUP BY RB.customer_id, C.first_name, C.last_name
ORDER BY total_balance DESC;
```

Key insights from sample data:

- Splitting joint account balances gives a more conservative estimate of customer value than assigning the full joint balance to each holder.
- After allocating joint balances, Sofia Almeida ranks highest at 11254.25 in estimated customer value.

Query 21 - Top Three Customers with Dynamic Holder Split

Business purpose: Calculate customer value by dividing each account balance by the number of holders, then return the top three customers.

SQL skills demonstrated: CTEs, COUNT, joins, balance allocation, LIMIT

```
-- Splits each account balance by the number of account holders and returns the top 3 customers.
WITH Holders AS (
    SELECT
        account_id,
        COUNT(customer_id) AS number_of_holders
    FROM account_holders
    GROUP BY account_id
)
SELECT
    AH.customer_id,
    ROUND(SUM(A.current_balance / H.number_of_holders), 2) AS allocated_total_balance
FROM accounts AS A
JOIN account_holders AS AH
    ON A.account_id = AH.account_id
JOIN Holders AS H
    ON AH.account_id = H.account_id
GROUP BY AH.customer_id
ORDER BY allocated_total_balance DESC
LIMIT 3;
```

Query 22 - Branch Performance Summary

Business purpose: Summarize how many customers and accounts are associated with each branch.

SQL skills demonstrated: COUNT DISTINCT, joins, GROUP BY

```
-- Shows core branch performance metrics: distinct customers and distinct accounts.
SELECT
    B.branch_id,
    B.branch_name,
    COUNT(DISTINCT C.customer_id) AS total_customers,
    COUNT(DISTINCT A.account_id) AS total_accounts
FROM branches AS B
JOIN customers AS C
    ON B.branch_id = C.home_branch_id
JOIN accounts AS A
    ON B.branch_id = A.branch_id
GROUP BY B.branch_id, B.branch_name;
```

Key insights from sample data:

- Kirchberg and Gare each serve three customers in the sample data, while Esch serves two.
- Kirchberg has the most accounts, giving it the strongest account footprint across the branch network.

Query 23 - Branch Deposit Netflow Draft

Business purpose: Summarize deposit netflow by branch and compare it across months.

SQL skills demonstrated: CTEs, branch-level aggregation, LAG

```
-- Draft: summarizes branch deposit netflow and prepares month-over-month comparison.
WITH BranchNetflow AS (
    SELECT
        B.branch_id,
        STRFTIME('%m', T.txn_date) AS month,
        COUNT(DISTINCT T.account_id) AS number_of_accounts,
        SUM(T.amount) AS deposit_netflow
    FROM transactions AS T
    JOIN accounts AS A
        ON T.account_id = A.account_id
    JOIN branches AS B
        ON A.branch_id = B.branch_id
    WHERE T.txn_type = 'deposit'
    GROUP BY B.branch_id, STRFTIME('%m', T.txn_date)
)
SELECT
    branch_id,
    month,
    number_of_accounts,
    deposit_netflow,
    LAG(deposit_netflow, 1) OVER (PARTITION BY branch_id ORDER BY month) AS prior_month_deposit_netflow
FROM BranchNetflow;
```

Query 24 - Customer Tenure and Netflow Segmentation

Business purpose: Segment customers by tenure and flag customers whose netflow is below the calculated benchmark.

SQL skills demonstrated: NTILE, CTEs, CASE, customer segmentation

```
-- Segments customers by tenure quartile and flags customers below the netflow benchmark.
WITH DaysJoined AS (
    SELECT
        customer_id,
        joined_date,
        ROUND(JULIANDAY(CURRENT_TIMESTAMP) - JULIANDAY(joined_date)) AS days_joined,
        NTILE(4) OVER (ORDER BY ROUND(JULIANDAY(CURRENT_TIMESTAMP) - JULIANDAY(joined_date))) AS quartile
    FROM customers
),
CustomerNetflow AS (
    SELECT
        D.customer_id,
```

```

        SUM(T.amount) AS netflow
    FROM transactions AS T
    JOIN account_holders AS AH
        ON T.account_id = AH.account_id
    JOIN DaysJoined AS D
        ON AH.customer_id = D.customer_id
    GROUP BY D.customer_id
),
AVGNetflow AS (
    SELECT
        AVG(netflow) AS bank_average_netflow
    FROM CustomerNetflow
)
SELECT
    D.customer_id,
    CASE
        WHEN D.quartile = 1 THEN 'New Customer'
        WHEN D.quartile IN (2, 3) THEN 'Value Customer'
        ELSE 'Veteran Customer'
    END AS customer_status,
    C.netflow,
    CASE
        WHEN C.netflow < AN.bank_average_netflow THEN 'At Risk'
        ELSE ''
    END AS risk_status
FROM DaysJoined AS D
JOIN CustomerNetflow AS C
    ON D.customer_id = C.customer_id
CROSS JOIN AVGNetflow AS AN
ORDER BY C.netflow DESC;

```

Query 25 - Large Transaction Review

Business purpose: Pull transactions where at least 1000 left the account in a single transaction.

SQL skills demonstrated: Filtering, risk review

```

-- Flags individual transactions where the outflow is 1000 or greater.
SELECT *
FROM transactions
WHERE amount <= -1000;

```

Query 26 - Monthly Negative Transaction Risk Flag

Business purpose: Flag accounts that had more than two negative transactions in a given month.

SQL skills demonstrated: CTE, COUNT, CASE, monthly grouping

```

-- Flags accounts with more than two outflow transactions in the same month.
WITH MonthlyOutflow AS (
    SELECT
        account_id,
        STRFTIME('%m', txn_date) AS month,
        COUNT(amount) AS negative_transaction_count
    FROM transactions
    WHERE amount < 0
    GROUP BY account_id, STRFTIME('%m', txn_date)
)
SELECT
    account_id,
    month,
    negative_transaction_count,
    CASE
        WHEN negative_transaction_count > 2 THEN 'Flag'
        ELSE ''
    END AS risk
FROM MonthlyOutflow;

```

Query 27 - Month-to-Month Transaction Count Change

Business purpose: Measure customer transaction count changes from one month to the next.

SQL skills demonstrated: CTEs, LAG, percentage calculation

```
-- Calculates the percentage change in monthly transaction count for each customer.
WITH MonthlyTxn AS (
    SELECT
        AH.customer_id,
        STRFTIME('%m', T.txn_date) AS month,
        COUNT(T.transaction_id) AS num_txn
    FROM transactions AS T
    JOIN account_holders AS AH
        ON T.account_id = AH.account_id
    GROUP BY AH.customer_id, STRFTIME('%m', T.txn_date)
),
MonthToMonth AS (
    SELECT
        *,
        LAG(num_txn, 1) OVER (PARTITION BY customer_id ORDER BY month) AS prev_mon_txn
    FROM MonthlyTxn
)
SELECT
    customer_id,
    month,
    num_txn,
    prev_mon_txn,
    (CAST(ROUND(((num_txn - prev_mon_txn) * 100.0 / prev_mon_txn), 2) AS TEXT) || '%') AS percent_change
FROM MonthToMonth
WHERE prev_mon_txn IS NOT NULL;
```

Query 28 - Customer Transaction History Summary

Business purpose: Show each customer's first and latest transaction dates, transaction count, and card payment count.

SQL skills demonstrated: MIN, MAX, COUNT, conditional aggregation, joins

```
-- Summarizes customer transaction history, including first/latest transaction and card payment count.
WITH TotalTransactions AS (
    SELECT
        C.customer_id,
        COUNT(T.transaction_id) AS total_txns,
        SUM(CASE WHEN T.txn_type = 'card_payment' THEN 1 ELSE 0 END) AS num_cardpayments
    FROM customers AS C
    JOIN account_holders AS AH
        ON C.customer_id = AH.customer_id
    JOIN transactions AS T
        ON AH.account_id = T.account_id
    GROUP BY C.customer_id
),
TxnHistory AS (
    SELECT
        AH.customer_id,
        MIN(T.txn_date) AS first_txn,
        MAX(T.txn_date) AS latest_txn
    FROM transactions AS T
    JOIN account_holders AS AH
        ON T.account_id = AH.account_id
    GROUP BY AH.customer_id
)
SELECT
    TT.customer_id,
    C.joined_date,
    TH.first_txn,
    TH.latest_txn,
    ABS(JULIANDAY(TH.first_txn) - JULIANDAY(TH.latest_txn)) AS days_between_first_and_latest_txn,
    TT.total_txns,
    TT.num_cardpayments
FROM TotalTransactions AS TT
JOIN customers AS C
```

```

    ON TT.customer_id = C.customer_id
JOIN TxnHistory AS TH
    ON C.customer_id = TH.customer_id;

```

Query 29 - Customer Health Flags

Business purpose: Create simple binary indicators for balance, savings, and recent activity conditions.

SQL skills demonstrated: CTEs, CASE, joins, flag design

```

-- Creates customer-level health flags for low balance, low savings balance, and recent activity.
WITH RecentTxns AS (
    SELECT
        C.customer_id,
        MAX(T.txn_date) AS max_txn_date
    FROM transactions AS T
    JOIN account_holders AS AH
        ON T.account_id = AH.account_id
    JOIN customers AS C
        ON AH.customer_id = C.customer_id
    GROUP BY C.customer_id
),
TotalBalance AS (
    SELECT
        C.customer_id,
        ROUND(SUM(A.current_balance), 2) AS balance
    FROM accounts AS A
    JOIN account_holders AS AH
        ON A.account_id = AH.account_id
    JOIN customers AS C
        ON AH.customer_id = C.customer_id
    GROUP BY C.customer_id
),
SavingsAccs AS (
    SELECT
        C.customer_id,
        A.account_type,
        A.current_balance
    FROM accounts AS A
    JOIN account_holders AS AH
        ON A.account_id = AH.account_id
    JOIN customers AS C
        ON AH.customer_id = C.customer_id
)
SELECT
    TB.customer_id,
    CASE WHEN TB.balance < 1000 THEN 1 ELSE 0 END AS low_total_balance_flag,
    CASE WHEN S.account_type = 'Savings' AND S.current_balance < 2000 THEN 1 ELSE 0 END AS
low_savings_balance_flag,
    CASE WHEN RT.max_txn_date > '2024-03-01' THEN 1 ELSE 0 END AS recent_activity_flag
FROM TotalBalance AS TB
JOIN RecentTxns AS RT
    ON TB.customer_id = RT.customer_id
JOIN SavingsAccs AS S
    ON TB.customer_id = S.customer_id;

```

Key insights from sample data:

- Elena Petrova is the clearest balance-risk example because her total balance is below zero.
- The flag structure can be expanded into a simple customer health score for retention, risk, or relationship-management reporting.

Appendix A - Database Schema

The following DDL creates the MeridianBank database structure used throughout the analysis.

```
PRAGMA foreign_keys = ON;

CREATE TABLE branches (
  branch_id    INTEGER PRIMARY KEY,
  branch_name  TEXT NOT NULL,
  city         TEXT NOT NULL,
  country      TEXT NOT NULL,
  opened_date  TEXT NOT NULL
);

CREATE TABLE merchants (
  merchant_id  INTEGER PRIMARY KEY,
  merchant_name TEXT NOT NULL,
  category     TEXT NOT NULL,
  country      TEXT
);

CREATE TABLE customers (
  customer_id  INTEGER PRIMARY KEY,
  first_name   TEXT NOT NULL,
  last_name    TEXT NOT NULL,
  email        TEXT NOT NULL UNIQUE,
  country      TEXT,
  joined_date  TEXT NOT NULL,
  home_branch_id INTEGER NOT NULL,
  FOREIGN KEY (home_branch_id) REFERENCES branches(branch_id)
);

CREATE TABLE accounts (
  account_id    INTEGER PRIMARY KEY,
  account_type  TEXT NOT NULL CHECK (account_type IN ('Checking','Savings','Credit')),
  branch_id     INTEGER NOT NULL,
  opened_date   TEXT NOT NULL,
  status        TEXT NOT NULL DEFAULT 'active'
  CHECK (status IN ('active','closed','frozen')),
  current_balance NUMERIC NOT NULL DEFAULT 0,
  FOREIGN KEY (branch_id) REFERENCES branches(branch_id)
);

CREATE TABLE account_holders (
```

```

account_id INTEGER NOT NULL,
customer_id INTEGER NOT NULL,
role       TEXT NOT NULL DEFAULT 'primary'
           CHECK (role IN ('primary','joint')),
PRIMARY KEY (account_id, customer_id),
FOREIGN KEY (account_id) REFERENCES accounts(account_id),
FOREIGN KEY (customer_id) REFERENCES customers(customer_id)
);

CREATE TABLE transactions (
  transaction_id INTEGER PRIMARY KEY,
  account_id     INTEGER NOT NULL,
  txn_date       TEXT NOT NULL,
  amount         NUMERIC NOT NULL,
  txn_type       TEXT NOT NULL CHECK (txn_type IN
    ('salary','deposit','withdrawal','transfer','card_payment','fee','interest')),
  merchant_id    INTEGER,
  FOREIGN KEY (account_id) REFERENCES accounts(account_id),
  FOREIGN KEY (merchant_id) REFERENCES merchants(merchant_id)
);

```

Appendix B - Seed Data

The following DML loads sample banking data used by the analysis queries.

```

-- Branches -----
INSERT INTO branches (branch_id, branch_name, city, country, opened_date) VALUES
(1, 'Kirchberg Branch', 'Luxembourg City', 'LU', '2015-03-01'),
(2, 'Gare Branch',      'Luxembourg City', 'LU', '2017-06-15'),
(3, 'Esch Branch',      'Esch-sur-Alzette', 'LU', '2019-09-10');
-- Merchants -----
INSERT INTO merchants (merchant_id, merchant_name, category, country) VALUES
(1, 'Cactus Supermarket', 'Groceries', 'LU'),
(2, 'CFL Trains',         'Transport', 'LU'),
(3, 'Amazon',             'Retail', 'LU'),
(4, 'Enovos Energy',      'Utilities', 'LU'),
(5, 'Brasserie Guillaume', 'Dining', 'LU'),
(6, 'Spotify',            'Subscription', 'LU');
-- Customers -----
INSERT INTO customers (customer_id, first_name, last_name, email, country, joined_date, home_branch_id) VALUES
(1, 'Amelie', 'Laurent', 'amelie.laurent@example.lu', 'LU', '2022-01-05', 1),
(2, 'Marco', 'Rossi', 'marco.rossi@example.lu', 'IT', '2022-01-08', 1),
(3, 'Sofia', 'Almeida', 'sofia.almeida@example.lu', 'PT', '2021-11-01', 2),
(4, 'Liam', 'OBrien', 'liam.obrien@example.lu', 'IE', '2023-03-18', 2),
(5, 'Yuki', 'Tanaka', 'yuki.tanaka@example.lu', 'JP', '2020-07-10', 3),
(6, 'Hassan', 'Khan', 'hassan.khan@example.lu', 'PK', '2022-08-25', 1),
(7, 'Elena', 'Petrova', 'elena.petrova@example.lu', 'BG', '2023-01-10', 3),
(8, 'Clara', 'Muller', 'clara.muller@example.lu', 'DE', '2021-06-28', 2);
-- Accounts (current_balance equals the sum of that account's ledger) ---
INSERT INTO accounts (account_id, account_type, branch_id, opened_date, status, current_balance) VALUES
(1, 'Checking', 1, '2022-01-10', 'active', 8532.55), -- joint: Amelie + Marco
(2, 'Savings', 1, '2022-02-01', 'active', 1503.80), -- Amelie
(3, 'Checking', 2, '2021-11-05', 'active', 6748.00), -- Sofia

```

```

(4, 'Checking', 2, '2023-03-20', 'active', 5014.51), -- Liam
(5, 'Savings', 3, '2020-07-15', 'active', 7016.50), -- Yuki
(6, 'Checking', 1, '2022-09-01', 'active', 9150.00), -- Hassan
(7, 'Credit', 3, '2023-01-12', 'active', -150.00), -- Elena (owes 150)
(8, 'Checking', 2, '2021-06-30', 'active', 6280.01), -- Clara
(9, 'Savings', 1, '2022-05-18', 'active', 9012.50); -- joint: Marco + Sofia
-- Account holders (the M:N links; accounts 1 and 9 are joint) -----
INSERT INTO account_holders (account_id, customer_id, role) VALUES
(1, 1, 'primary'), (1, 2, 'joint'),
(2, 1, 'primary'),
(3, 3, 'primary'),
(4, 4, 'primary'),
(5, 5, 'primary'),
(6, 6, 'primary'),
(7, 7, 'primary'),
(8, 8, 'primary'),
(9, 2, 'primary'), (9, 3, 'joint');
-- Transactions (the ledger) -----
INSERT INTO transactions (account_id, txn_date, amount, txn_type, merchant_id) VALUES
-- acc1 (Checking, Amelie + Marco)
(1, '2024-01-05', 3200.00, 'salary', NULL),
(1, '2024-01-08', -85.40, 'card_payment', 1),
(1, '2024-01-15', -52.00, 'card_payment', 2),
(1, '2024-01-20', -200.00, 'withdrawal', NULL),
(1, '2024-02-05', 3200.00, 'salary', NULL),
(1, '2024-02-10', -120.75, 'card_payment', 3),
(1, '2024-02-18', -64.30, 'card_payment', 5),
(1, '2024-03-05', 3200.00, 'salary', NULL),
(1, '2024-03-12', -45.00, 'card_payment', 1),
(1, '2024-03-25', -500.00, 'transfer', NULL),
-- acc2 (Savings, Amelie)
(2, '2024-02-01', 1000.00, 'deposit', NULL),
(2, '2024-03-01', 2.50, 'interest', NULL),
(2, '2024-03-25', 500.00, 'transfer', NULL),
(2, '2024-03-31', 1.30, 'interest', NULL),
-- acc3 (Checking, Sofia)
(3, '2024-01-03', 2500.00, 'salary', NULL),
(3, '2024-01-10', -200.00, 'card_payment', 3),
(3, '2024-01-22', -90.00, 'card_payment', 1),
(3, '2024-02-03', 2500.00, 'salary', NULL),
(3, '2024-02-14', -150.00, 'card_payment', 5),
(3, '2024-02-28', -12.00, 'fee', NULL),
(3, '2024-03-03', 2500.00, 'salary', NULL),
(3, '2024-03-18', -300.00, 'withdrawal', NULL),
-- acc4 (Checking, Liam)
(4, '2024-01-07', 1800.00, 'salary', NULL),
(4, '2024-01-19', -75.50, 'card_payment', 1),
(4, '2024-02-07', 1800.00, 'salary', NULL),
(4, '2024-02-20', -300.00, 'card_payment', 3),
(4, '2024-03-07', 1800.00, 'salary', NULL),
(4, '2024-03-15', -9.99, 'card_payment', 6),
-- acc5 (Savings, Yuki)
(5, '2024-01-15', 5000.00, 'deposit', NULL),
(5, '2024-02-01', 2000.00, 'deposit', NULL),
(5, '2024-02-15', 8.00, 'interest', NULL),
(5, '2024-03-15', 8.50, 'interest', NULL),

```

```

-- acc6 (Checking, Hassan) -- has two large movements (AML practice)
(6, '2024-01-04', 4100.00, 'salary', NULL),
(6, '2024-01-16', -220.00, 'card_payment', 4),
(6, '2024-01-28', -130.00, 'card_payment', 1),
(6, '2024-02-04', 4100.00, 'salary', NULL),
(6, '2024-02-20', -1500.00, 'transfer', NULL),
(6, '2024-03-04', 4100.00, 'salary', NULL),
(6, '2024-03-10', -300.00, 'card_payment', 3),
(6, '2024-03-22', -1000.00, 'withdrawal', NULL),
-- acc7 (Credit, Elena)
(7, '2024-01-12', -450.00, 'card_payment', 3),
(7, '2024-01-25', -80.00, 'card_payment', 5),
(7, '2024-02-12', 300.00, 'deposit', NULL),
(7, '2024-02-20', -200.00, 'card_payment', 1),
(7, '2024-03-12', 400.00, 'deposit', NULL),
(7, '2024-03-20', -120.00, 'card_payment', 2),
-- acc8 (Checking, Clara)
(8, '2024-01-06', 2200.00, 'salary', NULL),
(8, '2024-01-18', -60.00, 'card_payment', 1),
(8, '2024-02-06', 2200.00, 'salary', NULL),
(8, '2024-02-22', -9.99, 'card_payment', 6),
(8, '2024-03-06', 2200.00, 'salary', NULL),
(8, '2024-03-19', -250.00, 'card_payment', 3),
-- acc9 (Savings, Marco + Sofia)
(9, '2024-01-20', 3000.00, 'deposit', NULL),
(9, '2024-02-20', 3000.00, 'deposit', NULL),
(9, '2024-02-28', 5.00, 'interest', NULL),
(9, '2024-03-20', 3000.00, 'deposit', NULL),
(9, '2024-03-31', 7.50, 'interest', NULL);

```

Appendix C - Seed Data Notes

Key details preserved from the source file:

- Accounts 1 and 9 are joint accounts, supporting many-to-many account holder logic.
- Positive transaction amounts represent inflows, while negative amounts represent outflows.
- Card payments connect to merchants through merchant_id, which enables category spend analysis.
- Hassan's account includes large movements for AML/risk-style practice.